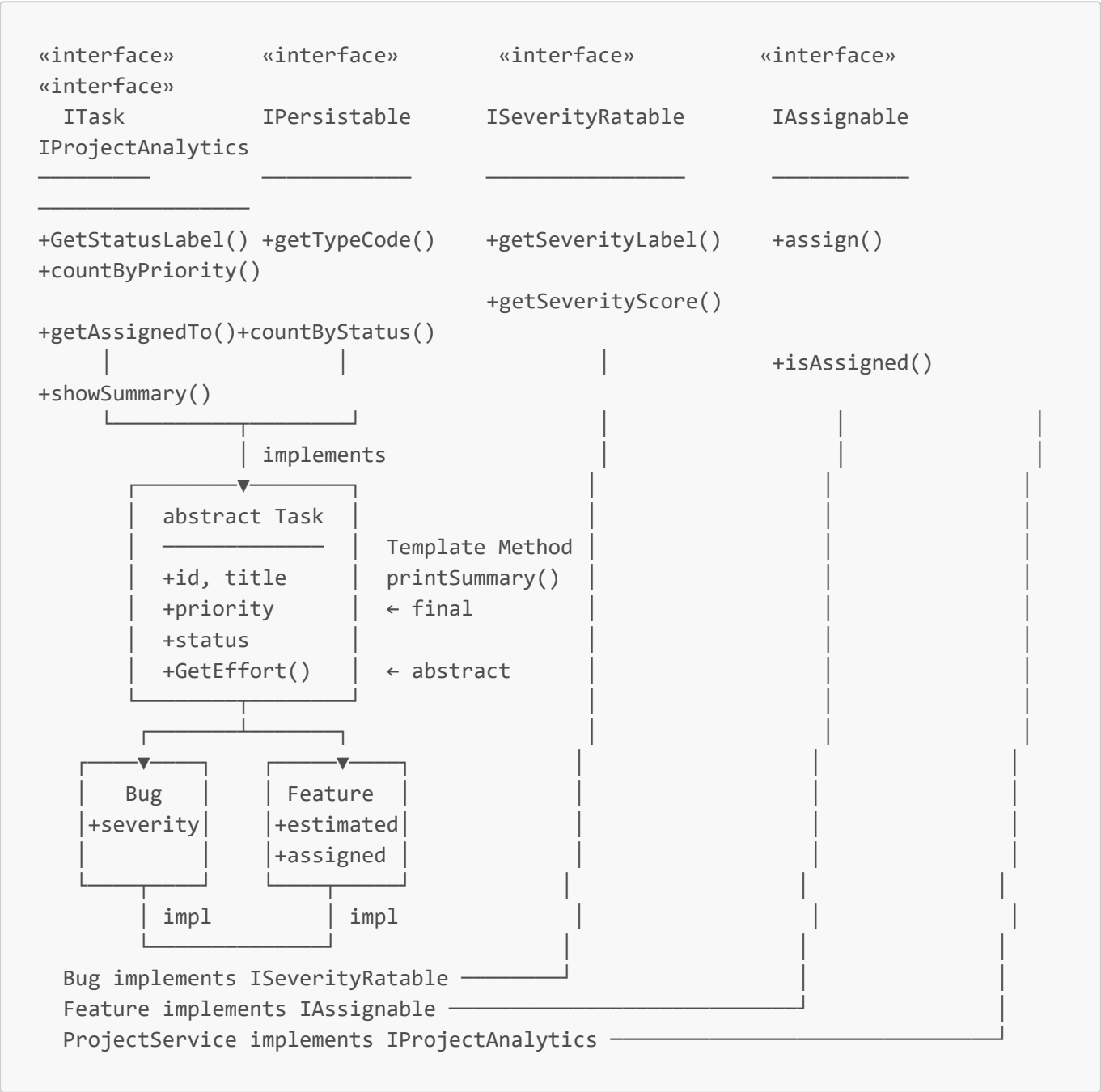


ProjectManagerApp — Hướng Dẫn 01: Core Models

Package: `com.projectmanager.models` · `com.projectmanager.models.entity` · `com.projectmanager.models.dto`

Sơ Đồ Kế Thừa & Interface



ITask.java

```
package com.projectmanager.models;
```

```
// Interface 1: hành vi cơ bản của mọi task trong hệ thống
// Task abstract implement – Bug và Feature kế thừa thông qua Task
public interface ITask {

    // Trả về nhãn trạng thái tiếng Việt (override bằng switch expression trong subclass)
    String getStatusLabel();
}
```

IPersistable.java

```
package com.projectmanager.models;

// Interface 2: đánh dấu object có thể lưu xuống DB
// Task abstract implement – Bug trả về "B", Feature trả về "F"
// TaskRepository dùng typeCode làm discriminator column khi INSERT và mapRow()
public interface IPersistable {

    // "B" = Bug | "F" = Feature – dùng trong TaskRepository.insert() và mapRow()
    String getTypeCode();
}
```

ISeverityRatable.java

```
package com.projectmanager.models;

// Interface 3: CHỈ Bug implement
// Feature không có severity – đây là điểm phân biệt Bug với Feature qua interface
public interface ISeverityRatable {

    // Nhãn mức nghiêm trọng bằng tiếng Việt (TaskListController dùng để hiển thị)
    String getSeverityLabel();

    // Điểm số để so sánh: LOW=1, MEDIUM=2, HIGH=3, CRITICAL=4
    int getSeverityScore();
}
```

IAssignable.java

```
package com.projectmanager.models;

// Interface 4: CHỈ Feature implement
// Bug không assign developer trực tiếp – Feature thì có
```

```
public interface IAssignable {

    void assign(String developerName); // gán developer
    String getAssignedTo();           // "" nếu chưa assign
    boolean isAssigned();              // true nếu đã assign
}
```

IProjectAnalytics.java

```
package com.projectmanager.models;

import java.util.Map;

// Interface 5: ProjectService<T> implement
// Tách biệt hành vi "phân tích dự án" – Dashboard dùng để hiển thị stats
public interface IProjectAnalytics {

    // {"LOW"→2, "MEDIUM"→3, "HIGH"→1} – dùng HashMap bên trong service
    Map<String, Integer> countByPriority();

    // {"todo"→3, "in_progress"→1, "done"→2}
    Map<String, Integer> countByStatus();

    // In tổng kết ra console (dùng cho debug, không dùng trên JavaFX UI)
    void showSummary();
}
```

Task.java (Abstract Base)

```
package com.projectmanager.models;

// Abstract class: implements ITask + IPersistable
// Template Method Pattern: printSummary() là method final định sẵn format
// – gọi GetEffort() và GetStatusLabel() mà subclass PHẢI cung cấp
public abstract class Task implements ITask, IPersistable {

    public String id;           // "B001", "F002"
    public String title;        // tên ngắn của task
    public String priority;     // "LOW" | "MEDIUM" | "HIGH"
    public String status;       // "todo" | "in_progress" | "done"

    // Effort tính khác nhau: Bug = severityScore×3h, Feature = estimatedHours
    public abstract int GetEffort();

    // Template Method Pattern – final: định format cột cố định, không cho override
    // Gọi getTypeCode() + GetStatusLabel() + GetEffort() – tất cả do subclass
```

```

cung cấp
    public final void printSummary() {
        System.out.printf("[%s] %-8s | %-35s | Pri:%-6s | %-22s | %dh%n",
            getTypeCode(), id, title, priority, GetStatusLabel(), GetEffort());
    }
}

```

Bug.java

```

package com.projectmanager.models;

// Bug: extends Task (ITask + IPersistable kế thừa)
//      implements ISeverityRatable – chỉ Bug có severity
public class Bug extends Task implements ISeverityRatable {

    public String severity; // "LOW" | "MEDIUM" | "HIGH" | "CRITICAL"

    @Override
    public String getTypeCode() { return "B"; }

    // ISeverityRatable
    @Override
    public String getSeverityLabel() {
        return switch (severity == null ? "" : severity) {
            case "LOW"      -> "Thap";
            case "MEDIUM"  -> "Trung binh";
            case "HIGH"    -> "Cao";
            case "CRITICAL" -> "Nghiem trong";
            default        -> "Chua xac dinh";
        };
    }

    @Override
    public int getSeverityScore() {
        return switch (severity == null ? "" : severity) {
            case "LOW"      -> 1;
            case "MEDIUM"  -> 2;
            case "HIGH"    -> 3;
            case "CRITICAL" -> 4;
            default        -> 0;
        };
    }

    // GetEffort: Bug effort tính theo severity – LOW=3h, MEDIUM=6h, HIGH=9h,
    // CRITICAL=12h
    @Override
    public int GetEffort() { return getSeverityScore() * 3; }

    // ITask – switch expression (Modern Java, bài 20 yêu cầu)

```

```
// Prefix "[BUG]" để phân biệt khi in danh sách chung Bug + Feature
@Override
public String GetStatusLabel() {
    return switch (status == null ? "" : status) {
        case "todo"      -> "[BUG] Chưa xử lý";
        case "in_progress" -> "[BUG] Đang sửa";
        case "done"       -> "[BUG] Đã sửa xong";
        default          -> "[BUG] Không xác định";
    };
}
}
```

Feature.java

```
package com.projectmanager.models;

// Feature: extends Task (ITask + IPersistable kế thừa)
//           implements IAssignable – chỉ Feature có thể assign developer
public class Feature extends Task implements IAssignable {

    public int    estimatedHours;          // số giờ ước tính

    // private: chỉ truy cập qua IAssignable methods – encapsulation
    private String assignedTo = "";

    @Override
    public String getTypeCode() { return "F"; }

    // IAssignable
    @Override
    public void assign(String developerName) {
        this.assignedTo = (developerName == null) ? "" : developerName.trim();
    }

    @Override
    public String getAssignedTo() { return assignedTo; }

    @Override
    public boolean isAssigned() { return assignedTo != null &&
!assignedTo.isBlank(); }

    // Feature effort = estimatedHours nhập trực tiếp
    @Override
    public int GetEffort() { return estimatedHours; }

    // ITask – switch expression với prefix "[FEAT]"
    @Override
    public String GetStatusLabel() {
        return switch (status == null ? "" : status) {
```

```
        case "todo"          -> "[FEAT] Chưa bắt đầu";
        case "in_progress" -> "[FEAT] Đang phát triển";
        case "done"         -> "[FEAT] Đã hoàn thành";
        default             -> "[FEAT] Không xác định";
    };
}
}
```

User.java (Entity)

```
package com.projectmanager.models.entity;

// Entity ánh xạ bảng Users trong DB
// Không implements ITask hay IPersistable – User là domain khác (Auth, không phải Task)
public class User {

    public int      id;
    public String   username;
    public String   passwordHash; // SHA-256 Base64 – không lưu plain text
    public String   email;
    public String   role;          // "admin" hoặc "user"
    public boolean  status;        // true = active, false = blocked
}
```

LoginRequest.java (DTO)

```
package com.projectmanager.models.dto;

// DTO (Data Transfer Object): đóng gói dữ liệu đầu vào từ LoginController → AuthService
// Tách biệt input data với entity User – User là DB model, LoginRequest là form input
public class LoginRequest {

    public final String username;
    public final String password; // plain text – chỉ tồn tại trong request, không lưu DB

    public LoginRequest(String username, String password) {
        this.username = username;
        this.password = password;
    }
}
```

Ghi Chú Thiết Kế

Điểm	Giải thích
<code>Task</code> implements <code>ITask</code> , <code>IPersistable</code>	Một abstract class cam kết 2 contract: hành vi (<code>ITask</code>) + lưu trữ (<code>IPersistable</code>)
<code>Bug</code> implements <code>ISeverityRatable</code>	Chỉ Bug có severity — nếu để trong Task thì Feature sẽ phải có field không dùng đến
<code>Feature</code> implements <code>IAssignable</code>	Chỉ Feature assign developer — "fat interface" nếu gộp vào <code>ITask</code> chung
<code>GetEffort()</code> abstract trong Task	Bug tính $\text{severity} \times 3$; Feature dùng <code>estimatedHours</code> — logic khác nhau hoàn toàn
<code>printSummary()</code> là <code>final</code>	Template Method — format không thay đổi, chỉ nội dung (<code>GetEffort/GetStatusLabel</code>) thay đổi
<code>private assignedTo</code> trong Feature	Encapsulation — chỉ sửa qua <code>assign()</code> , đọc qua <code>getAssignedTo()</code>
<code>User</code> không implements <code>ITask/IPersistable</code>	User thuộc Auth domain, Task thuộc Project domain — tách biệt rõ ràng
<code>LoginRequest</code> final fields	Immutable DTO — sau khi tạo không thể sửa username/password

So Sánh Bug vs Feature qua Interfaces

	Bug	Feature
<code>getTypeCode()</code>	"B"	"F"
<code>ITask</code>	✓ (kế thừa qua Task)	✓ (kế thừa qua Task)
<code>IPersistable</code>	✓ (kế thừa qua Task)	✓ (kế thừa qua Task)
<code>ISeverityRatable</code>	✓ severity: LOW→CRITICAL	X (không có severity)
<code>IAssignable</code>	X (không assign dev)	✓ assignedTo: String
<code>GetEffort()</code>	$\text{severityScore} \times 3\text{h}$	<code>estimatedHours</code> trực tiếp
<code>GetStatusLabel()</code>	"[BUG] ..."	"[FEAT] ..."

Phân Loại Màu Trong UI (TaskListController)

Giá trị	Màu hex	Dùng ở đâu
Type = Bug	#F38BA8	Cột Type trong TableView
Type = Feature	#89DCEB	Cột Type trong TableView
Priority HIGH	#F38BA8	Cột Priority
Priority MEDIUM	#FAB387	Cột Priority

Giá trị	Màu hex	Dùng ở đâu
Priority LOW	#A6E3A1	Cột Priority
Status todo	#6C7086	Cột Status
Status in_progress	#FAB387	Cột Status
Status done	#A6E3A1	Cột Status